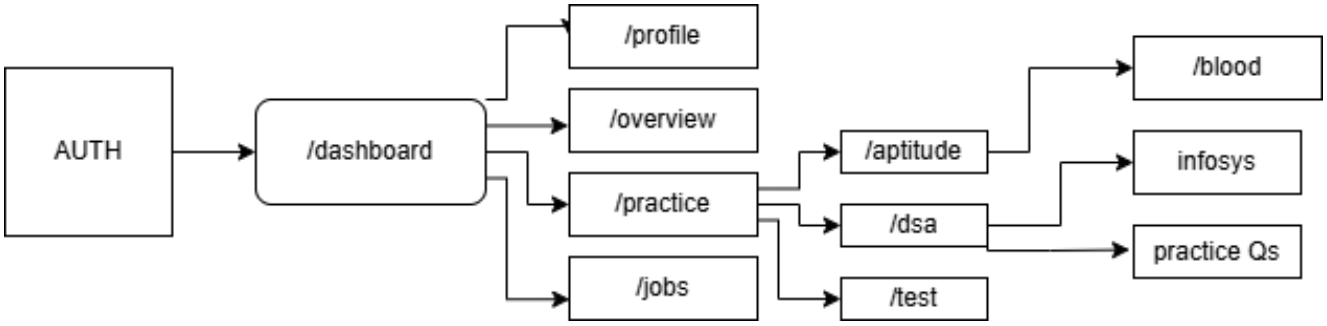
PHASE 1 - USER SYSTEM

FRONTEND PART

ARCHITECTURE

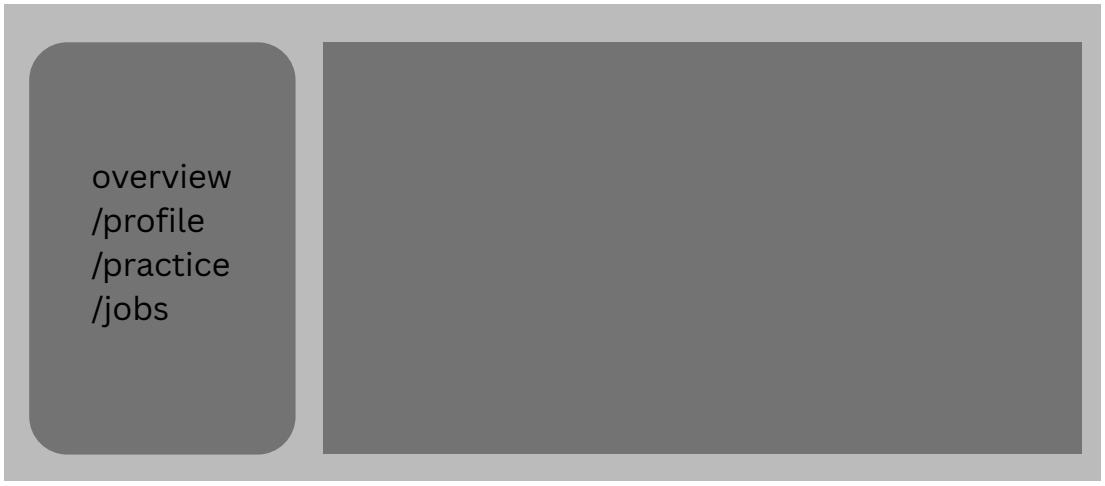


/auth

/dashboard

overview
/profile
/practice
/jobs

/practice/aptitude
/practice/dsa
/practice/test



Folder Structure

✅📁 src/ — Professional Folder Purpose Breakdown

📁 assets/ — Static visual files.
Stores non-code files used in UI.
📌 Why important?
Keeps your project organized and prevents mixing images with code.
Examples:

- images/logo.png
- icons/edit.svg

📁 components/ — Reusable UI building blocks.
Global or feature-specific components used across pages.
📌 Why important?
Prevents rewriting UI pieces; promotes reusability and cleaner code.
Examples:

- common/Navbar.jsx
- ui/Button.jsx
- dashboard/StatsCard.jsx

📁 config/ — App-level constants & configuration.
Central place for URLs, regex, navigation links, global settings.
📌 Why important?
Makes code clean, avoids magic values scattered everywhere.
Examples:

- API_BASE_URL
- sidebar navLinks array

📁 hooks/ — Custom reusable React logic.
Functions starting with use that encapsulate logic.
📌 Why important?
Keeps components clean; avoids repeating logic in multiple pages.
Examples:
useAuth() — handles login state
useFetch() — reusable API calling hook
useDebounce() — optimized search input

📁 layouts/ — Structural page wrappers (frames).
Define the outer skeleton of screens.
📌 Why important?
Avoids repeating layout patterns across pages.
Examples:
DashboardLayout.jsx → Sidebar + Header + Outlet
AuthLayout.jsx → Centered login/signup layout

📁 pages/ — Screens connected to routes.
Each page is a full screen in your app.
📌 Why important?
Routing stays clean; each page is feature-isolated.
Examples:
/auth/Login.jsx
/dashboard/Overview.jsx
/dashboard/practice/TestScreen.jsx

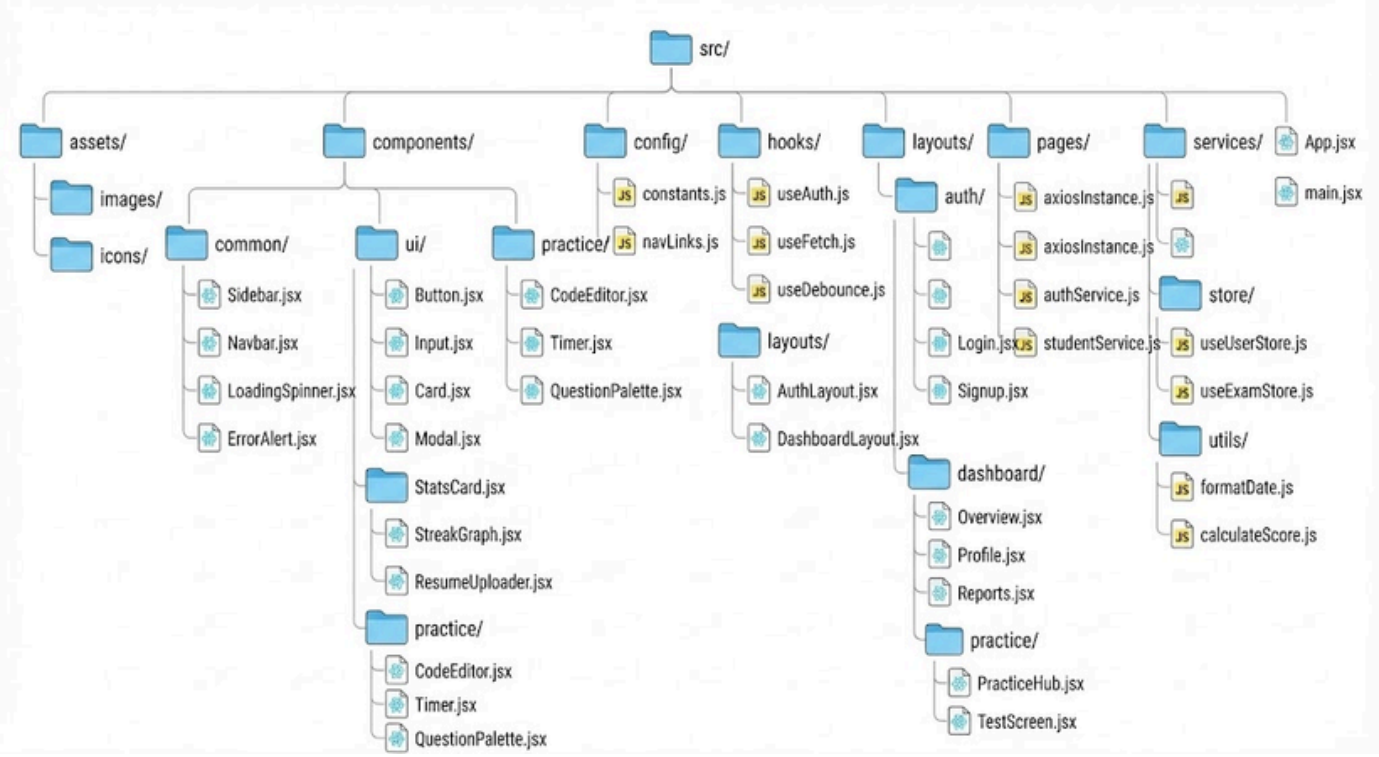
📁 services/ — API layer + backend communication.
Contains all external network calls using Axios/fetch.
📌 Why important?
✔ Keeps API logic separate from UI
✔ Easy to maintain/replace backend
✔ Cleaner components (UI = UI, Logic = services)
Examples:
authService.login()
studentService.fetchReports()
axiosInstance.js → attaches JWT automatically

📂 store/ — Global state management (Zustand/Redux).
Holds app-wide data that many components rely on.
📌 Why important?
Prevents prop-drilling and makes state predictable.
Examples:
useUserStore.js — user details
useExamStore.js — exam state, autosave answers

📂 utils/ — Pure utility functions (no React).
Independent helper functions used anywhere.
📌 Why important?
✔️ Speeds up development
✔️ Avoids re-writing logic
✔️ Keeps React files clean
Examples:
formatDate("2025-10-12")
calculateScore(answers)
generateUsername()
(If it doesn't need hooks, state, UI → it belongs in utils.)

📄 App.jsx — Root app structure + router.
Contains all routes and layouts in one place.
📌 Why important?
Provides a clean starting point for the entire UI.

📄 main.jsx — Entry point for React.
Bootstraps the app and renders it to the DOM.
📌 Why important?
Required for initializing React + providers.


src/
assets/
images/
icons/
components/
common/
Sidebar.jsx
Navbar.jsx
LoadingSpinner.jsx
ErrorAlert.jsx
ui/
Button.jsx
Input.jsx
Card.jsx
Modal.jsx
StatsCard.jsx
StreakGraph.jsx
ResumeUploader.jsx
practice/
CodeEditor.jsx
Timer.jsx
QuestionPalette.jsx
practice/
CodeEditor.jsx
Timer.jsx
QuestionPalette.jsx
config/
constants.js
navLinks.js
hooks/
useAuth.js
useFetch.js
useDebounce.js
layouts/
AuthLayout.jsx
DashboardLayout.jsx
layouts/
auth/
Login.jsx
Signup.jsx
dashboard/
Overview.jsx
Profile.jsx
Reports.jsx
practice/
PracticeHub.jsx
TestScreen.jsx
pages/
axiosInstance.js
axiosInstance.js
authService.js
studentService.js
services/
store/
useUserStore.js
useExamStore.js
utils/
formatDate.js
calculateScore.js
App.jsx
main.jsx